SOFTWARE DEVELOPMENT PLAN

CareTrack

A personal appointment and medication tracking web application

Team 2

Project summary

CareTrack helps people keep doctor appointments, record the medication plan they received from a health professional, and review their tracking history in one place. The first version will focus on clear records, simple use, and basic privacy. It will not give medical advice or change a treatment plan.

1. Problem and Project Context

People may need to keep track of doctor appointments and medication times at the same time. Details can be spread across paper notes, phone calendars, and messages. This can make it hard to see past appointments or remember what needs to be discussed at the next visit.

CareTrack is a small web application that keeps these records together. A user can add appointments, save a medication schedule based on instructions from a health professional, and record whether a planned dose was taken, skipped, or postponed. The app stores what the user enters. It does not decide what medication a person should take.

Problem evidence and assumptions

- Initial assumption: some users prefer one place for appointment details and personal medication records. The team will check this with short interviews or a small survey before finalising the features.
- The first release is a student project prototype. It will use sample data and will not be presented as a medical device or a replacement for professional care.

Intended outcome

The intended outcome is a clear and easy-to-use record system. Users should be able to find upcoming appointments, review their entered medication plan, and look back at their own tracking history without searching through several places.

Initial project goals

Goal	How we will check it
Record appointments	A user can create, view, edit, and delete an appointment.
Record medication plans	A user can save a plan and view its planned times.
Keep a tracking history	A user can add a status record for a planned dose and review past records.
Protect user records	A signed-in user cannot access another user's records.

2. Stakeholders and Users

Group	Goals	Needs or limits
Main user / patient	Keep appointments and personal medication records in one place.	The app must be clear, quick to use, and easy to correct when an entry is wrong.
Family caregiver (optional)	Help a family member keep track of appointments when invited.	Must have permission before seeing shared information. Not every user wants to share records.
Project team	Build a working and testable student project by Week 12.	Time and experience with web development are limited.
Course instructor	Review the plan, code, tests, and team process.	The team must explain its choices and make the repository accessible.

Possible conflict: a caregiver may want more information, while the user may want privacy. The first version will keep sharing optional and limited. If permission-based sharing takes too much time, it will be left out of the first release.

3. Scope

In scope for the first release

- User registration, sign-in, and sign-out.
- Create, view, edit, and delete doctor appointments.
- Save a medication plan entered by the user from instructions given by a health professional.
- Show planned medication times and let the user record a status: taken, skipped, or postponed.
- Show a simple history of appointments and tracking records.
- Basic input checks and access control so users can only view their own private records.
- A simple dashboard with upcoming appointments and planned medication times.

Out of scope for the first release

- Medical advice, diagnosis, treatment suggestions, or dose calculations.
- Changing or recommending medication instructions.
- Connecting to hospital systems, pharmacy systems, or national health records.
- Automatic sharing with family members or healthcare staff.
- Mobile apps, wearable devices, payment features, or video appointments.
- Guaranteeing notifications when the browser or device is closed.

Initial success criteria

ID	Success criterion	Evidence
SC1	A user can sign in and only access their own records.	Access-control tests with two test accounts.
SC2	Appointments can be created, updated, viewed, and deleted.	Test cases for each operation.
SC3	Medication plans and tracking statuses are saved correctly.	Database checks and form tests.
SC4	Invalid or missing form fields show a clear message.	Validation tests.
SC5	At least 4 of 5 test users can complete key tasks without help.	Short usability test with sample data.

Assumptions and dependencies

- Users enter correct details and follow the plan given by their health professional. The app does not verify medical instructions.

- The project will be built as a web app with Python, Django, HTML/CSS, and SQLite. The team will review this choice after a small technical test.
- The project needs a shared GitHub repository and a development environment on each team member's computer.

4. Initial Process and Work Plan

Process model

The team will use a simple iterative process. Work will be split into small tasks. Each task will be planned, built, reviewed, and tested before it is marked as done. The team will show a working version often and change the plan if a feature is too large or does not help the main goal.

Team roles and responsibilities

Role	Main tasks	Assigned member
Backend & Data Architect	Build the server side with Python (Django), set up the database schema, integrate the SMS notification API, and create the summary report algorithms that organise patient data for doctors.	Eren Korkut
Frontend Developer & UI/UX Designer	Design and build a simple, high-contrast web interface with high accessibility for patients over 65 (HTML/CSS/JS), and connect the interface to the backend API so data flows between both sides.	Rabia Hatice Beyhan
QA (Test) Engineer & Scrum Master	Track task distribution and sprints with the Kanban board on GitHub or Trello. Test time and notification deviations, interface buttons, and the end-to-end (E2E) flow of the system. Organise the project documentation.	Osman Berk Cerit

Roles can be shared or changed based on team size. Every member should review the main features and be able to explain how the project works.

Milestones through Week 12

Week	Planned work	Output
4	Agree on the problem, users, scope, risks, and technology.	SDP and initial diagrams
5	Set up GitHub, project board, Django project, and database.	App runs locally
6	Build sign-in and basic page layout.	Login and dashboard
7	Build appointment create/view/edit/delete features.	Appointment module
8	Build medication plan and planned times.	Medication plan module
9	Build status logs and history page.	Tracking history
10	Add input validation and access-control checks.	Security and form tests
11	Run usability tests, fix bugs, and update documentation.	Test report and improved app
12	Final checks, demo, and submission.	Working prototype and final report

Communication and decisions

- Hold one short team meeting each week and use the project board to track tasks.
- Use GitHub issues for bugs and tasks. Use pull requests for code changes.
- A change that affects scope, safety, or the data model must be discussed by the team and recorded in the project notes.
- If a feature threatens the deadline, keep the core features and move the extra feature out of scope.

5. Technology and Architecture Direction

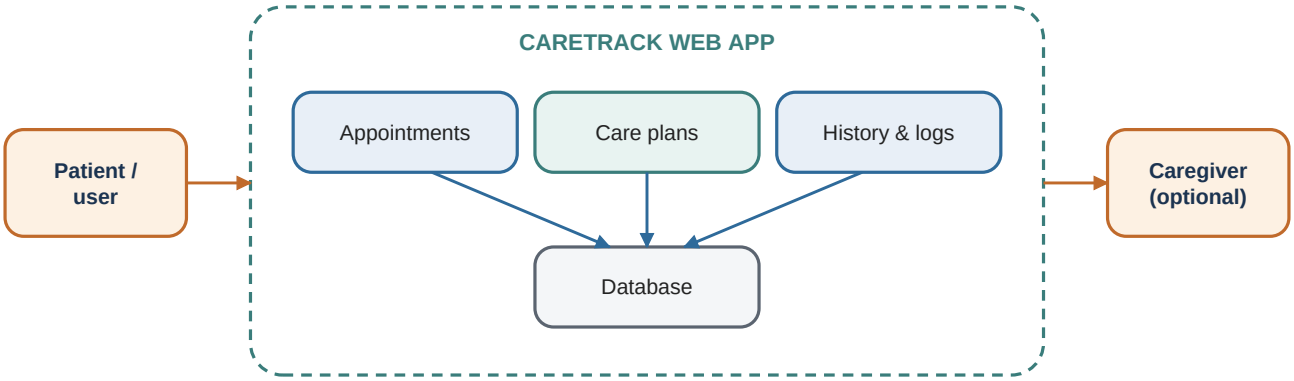
Proposed technology stack

Technology	Use	Reason and trade-off
Python	Main programming language	Readable and suitable for a first web project. The team still needs to learn its basics.
Django	Web framework	Includes tools for URL handling, forms, database models, and user accounts. It has concepts the team must learn.
HTML and CSS	Web pages and layout	Enough for a simple first interface. Complex interactive features may need JavaScript later.
SQLite	Development database	Easy to set up for a student prototype. A larger deployed service may need another database.
Git and GitHub	Version control and team work	Supports branches, code review, issue tracking, and a shared project history.

The team will avoid adding a separate frontend framework in the first release. Django templates with HTML and CSS should be enough for the planned features. This keeps the number of new tools lower.

Initial context / system-boundary diagram

CareTrack system context



Users manage their own records. Sharing is optional and permission-based.

Quality attributes

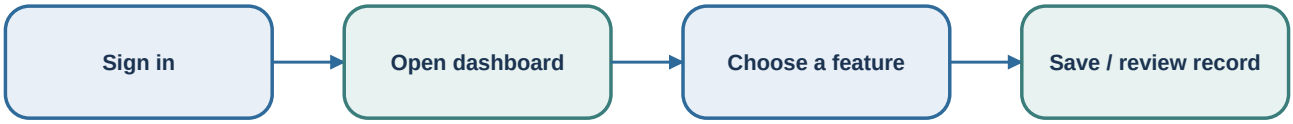
Attribute	Why it matters	Initial approach
Security and privacy	Appointment and medication records are private.	Require sign-in, check ownership on every record operation, and use sample data during development.
Usability	Users may have different levels of digital skill.	Use clear labels, short forms, readable text, and helpful error messages.
Reliability	Records should be saved and shown correctly.	Validate inputs and test create, edit, delete, and history actions.
Maintainability	The team needs to fix and extend the code during the project.	Keep models, views, templates, and tests organised; review changes through pull requests.

6. UX or Interaction Evidence

The first version will use a simple web layout. The dashboard will show the next appointments and planned medication times. Main actions will be available from a clear menu. The forms will ask only for information needed for the selected task.

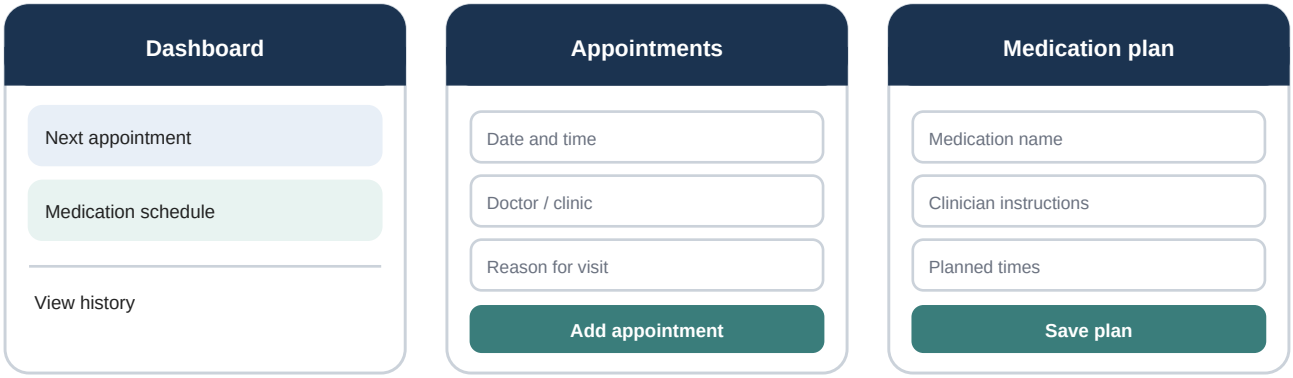
Initial user flow

Main user flow



Invalid entries show an error so the user can correct them.

Planned screen sketches



These are early layout sketches, not final designs. The team will test them with a few people and make changes based on feedback.

Key interaction rules

- Show a clear success message after saving a record.
- Ask the user to confirm before deleting a record.
- Show a clear error when a required field is missing or a date is invalid.
- Do not display another user's records. Do not use real health data in the class demo.

7. Repository and Collaboration Setup

Repository

GitHub URL:

https://github.com/ernkrkt10/akilli-takip-sistemi.git

The repository should be accessible to the instructor. Do not commit passwords, secret keys, or real health information.

Branch and pull-request workflow

- Keep the main branch stable. Do not push unfinished work directly to main.
- Create a short branch for each task, for example feature/appointment-form or test/login-access.
- Open a pull request when the task is ready. Include a short summary and how it was tested.
- At least one other team member reviews the change before it is merged.
- Resolve conflicts together and keep the issue or project board updated.

Initial README content

README section	Content to include
Project overview	CareTrack is a web app for personal appointment and medication tracking.
Main features	Appointments, user-entered medication plans, status logs, and history.
Technology	Python, Django, HTML/CSS, SQLite.
Setup instructions	Add tested steps for installing dependencies, running migrations, and starting the local server.
Testing	Explain how to run the project's tests.
Safety note	The app does not provide medical advice or change medication instructions. Use sample data only.

Project board

Use a simple board with the columns **Backlog**, **To do**, **In progress**, **Review**, and **Done**. Each task should have an owner and a clear completion check.

8. Initial Risk Register

Risk	Likelihood	Impact	Early warning	Mitigation	Contingency
Learning Django takes longer than planned.	High	High	Basic pages are not running by Week 6.	Use short tutorials and build one feature at a time.	Reduce optional features and focus on core records.
AI-generated code has errors or security gaps.	Medium	High	Tests fail or users can access records they do not own.	Review code, add tests, and check access rules.	Disable the affected feature until fixed.
Project scope grows too much.	Medium	High	New features keep being added before core tasks work.	Use the in-scope list and agree on changes.	Move extra features to a future version.
Team members conflict with code changes.	Medium	Medium	Frequent merge conflicts or unclear task ownership.	Use branches, pull requests, and small changes.	Pair up to resolve conflicts and restore a working version.
Private health data is exposed.	Low	High	Real data or secret keys appear in the repository or demo.	Use sample data, protect secrets, and test permissions.	Remove exposed data, rotate secrets, and inform the instructor.
The app does not meet user needs.	Medium	Medium	Test users cannot find key actions.	Ask a few people to review sketches and test tasks.	Simplify the layout and prioritise common tasks.

9. Responsible AI-Use Plan

The team may use AI tools as support during planning, coding, testing, and writing. AI output will be treated as a draft, not as a final answer. Team members remain responsible for the code and the submitted work.

Activities where AI may be used

- Explain unfamiliar Python or Django concepts and suggest small code examples.
- Help draft test cases, sample data, wireframe ideas, and documentation.
- Suggest ways to find bugs or improve code structure.
- Review wording in the report for clarity and simple English.

Review and testing of generated code or text

- A team member will read each AI-generated code change and explain what it does before merging it.

- The team will run the app and test the related feature. Important actions will have repeatable test cases.
- AI suggestions about security will be checked against the actual code and tests.
- The team will edit generated report text so it matches the work that was really done. No test or result will be claimed unless it happened.

Dependencies, licenses, and sensitive data

- Use known packages from trusted sources and record dependencies in the project files.
- Check package licenses and avoid copying code when its source or license is unclear.
- Never send passwords, secret keys, personal health records, or other private data to an AI tool.
- Use fictional sample users and sample medication entries in development, tests, and demos.

Documentation of important AI-assisted decisions

For major AI-assisted decisions, the team will keep a short note in the project documentation or issue tracker. The note will include the decision, why AI help was used, what the team checked, and whether the suggestion was accepted or changed. The team will also track important AI-generated code changes in pull requests.

Limits for this project

CareTrack will only store and show information entered by the user. It will not suggest a medicine, calculate a dose, or tell a user to start or stop treatment. A user should follow instructions from a qualified health professional. The prototype is for a course project and is not intended for real clinical use.

Final submission checklist

- Fill in the member names in the roles table and the GitHub URL.
- Confirm that the repository and project board are accessible to the instructor.
- Check that the final scope, technology choices, diagrams, and risks match the actual plan.
- Make sure all team members contributed and can explain the submission.
- Use the required filename format: Team2_D1_SDP.pdf.

End of initial plan. Changes that affect scope or technical direction should be recorded and explained.